实训 8

SpringBoot3 集成 MyBatis

实验步骤与完整代码

课程：JavaWeb 实训

项目：员工信息管理系统（employee 增删改查）

姓名：_____ 学号：_____

日期：2026 年 10 月 29 日

目 录

0. 先看全貌 3
1. 环境准备 3
2. 新建 Spring Boot 项目 4
3. 建数据库和 employee 表 6
4. 配置 application.yml 7
5. 实体类 Employee 7
6. 统一返回类 Result 9
7. Mapper 接口 10
8. Mapper XML 11
9. Service 业务层 12
10. Controller 控制层 13
11. 启动并测试 selectAll 15
12. 单个查询 16
13. 分页查询 17
14. 增删改操作（任务 4） 18
15. 完整验收清单 20
16. 常见报错排查 20
17. 讲义核心代码解释（复习用） 21
18. 提交前检查 23

0. 先看全貌

这次实验要完成的是**员工信息表 `employee` 的增删改查**，最终提供下面这些接口：

序号	请求方式	接口地址	作用
1	GET	/employee/selectAll	查询全部员工
2	GET	/employee/selectById/{id}	按 id 查询（路径参数）
3	GET	/employee/selectOne?id=1	按 id 查询（@Select 注解方式）
4	GET	/employee/selectPage?pageNum=1&pageSize=2	分页查询
5	POST	/employee/insert	新增员工
6	PUT	/employee/update	修改员工
7	DELETE	/employee/delete/{id}	删除员工

最终项目结构：

```
实训 8_SpringBoot3 集成 MyBatis/
├─ db/
│  └─ init.sql
├─ springboot/
│  ├─ pom.xml
│  └─ src/main/
│     ├─ java/com/example/
│     │  ├─ SpringbootApplication.java
│     │  ├─ common/Result.java
│     │  ├─ entity/Employee.java
│     │  ├─ mapper/EmployeeMapper.java
│     │  ├─ service/EmployeeService.java
│     │  ├─ controller/EmployeeController.java
│     │  └─ exception/GlobalExceptionHandler.java
│     └─ resources/
│        ├─ application.yml
│        └─ mapper/EmployeeMapper.xml
├─ requests.http
└─ README.md
```

> 讲义里的顺序是先写 EmployeeController，这时因为 Employee、EmployeeService 还没建，IDEA 会爆红。**爆红是正常的**，不是代码写错。
>
> 下面按“先建底层、再建上层”的顺序整理：数据库 → 实体 → Mapper → XML → Service → Controller。内容和讲义完全一致，只是顺序更容易一次跑通。

1. 环境准备

工具	要求	检查命令
JDK	21	java -version
Maven	3.8+	mvn -v
MySQL	8.x	mysql -uroot -p123456
IDEA	2025 / 2026 均可	能打开 Maven 项目
MyBatisX 插件	可选，推荐装	IDEA → Settings → Plugins → 搜索 MyBatisX

对应讲义：`5.` 安装插件。

装 MyBatisX 的作用：Mapper 接口和 XML 之间会出现小鸟图标，点一下可以互相跳转，避免方法名和 `id` 写错。

---

2. 新建 Spring Boot 项目

对应讲义：二、配置 `MyBatis` 开头部分。

2.1 IDEA 新建项目

1. 打开 IDEA，点击 **File → New → Project**。
2. 左侧选择 **Spring Boot**。
3. 填写：
    - Name：`employee-crud`
    - Group：`com.example`
    - Artifact：`employee-crud`
    - Type：`Maven`
    - Language：`Java`
    - Java version：`21`
4. 下一步，勾选依赖：
    - **Spring Web**
    - **MyBatis Framework**
    - **MySQL Driver**
5. 点击 **Create**。

如果不想从零新建，可以直接打开已经整理好的项目：

```
D:\course\Javaweb\实训 8_SpringBoot3 集成 MyBatis\springboot
```

IDEA 打开 `springboot` 这一层即可，它本身就是一个 Maven 项目。

2.2 确认 pom.xml

对应讲义：五、分页查询 → 1. 在项目的 `pom.xml` 文件里，引入分页插件。

完整 `pom.xml`：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/mav
en-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.16</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>employee-crud</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>employee-crud</name>
    <description>Spring Boot 3 + MyBatis 员工增删改查练习</description>

    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <!-- Web -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- MyBatis -->
        <dependency>
            <groupId>org.mybatis.spring.boot</groupId>
            <artifactId>mybatis-spring-boot-starter</artifactId>
            <version>3.0.5</version>
        </dependency>

        <!-- MySQL 驱动 -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- PageHelper 分页插件 -->
        <dependency>
            <groupId>com.github.pagehelper</groupId>
            <artifactId>pagehelper-spring-boot-starter</artifactId>
            <version>2.1.0</version>
            <exclusions>
                <exclusion>
                    <groupId>org.mybatis</groupId>
                    <artifactId>mybatis</artifactId>
                </exclusion>
            </exclusions>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

完成后点击 IDEA 右上角的 Maven 刷新按钮，确认依赖全部下载成功。

> **Maven 坐标是什么？**
>
> groupId + artifactId + version 三个值可以唯一确定一个 jar 包。比如 com.github.pagehelper:pagehelper-spring-boot-starter:2.1.0。在 pom.xml 里写坐标，Maven 就会自动去仓库下载。

---

3. 建数据库和 employee 表

对应讲义：9．在你连接的数据库中，新建数据库表:employee、添加几条记录。

3.1 建库建表 SQL

完整文件：db/init.sql

```sql
-- 数据库名是 xm-pro（中划线），SQL 中必须用反引号包起来
CREATE DATABASE IF NOT EXISTS `xm-pro`
    DEFAULT CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE `xm-pro`;

CREATE TABLE IF NOT EXISTS `employee` (
    `id`          INT         NOT NULL AUTO_INCREMENT COMMENT '主键',
    `name`        VARCHAR(255)         DEFAULT NULL COMMENT '姓名',
    `sex`         VARCHAR(255)         DEFAULT NULL COMMENT '性别',
    `no`          VARCHAR(255)         DEFAULT NULL COMMENT '工号',
    `age`         INT                  DEFAULT NULL COMMENT '年龄',
    `description` LONGTEXT             DEFAULT NULL COMMENT '个人介绍',
    PRIMARY KEY (`id`)
) ENGINE = InnoDB
  DEFAULT CHARSET = utf8mb4
  COLLATE = utf8mb4_unicode_ci COMMENT ='员工信息表';

-- 只有表为空时才插入示例数据，重复执行不会重复插入
INSERT INTO `employee` (`name`, `sex`, `no`, `age`, `description`)
SELECT t.name, t.sex, t.no, t.age, t.description
FROM (
    SELECT 'thc' AS name, '男' AS sex, '100001' AS no, 18 AS age, '勤学钻研，自强不息' AS descrip
tion
    UNION ALL
    SELECT 'zhangsan', '男', '100002', 19, '积极开拓，敢于创新'
    UNION ALL
    SELECT 'lisi', '女', '100003', 23, '做事细致严谨'
) t
WHERE NOT EXISTS (SELECT 1 FROM `employee`);
```

执行方式（PowerShell 或 CMD）：

```
mysql -uroot -p123456 --default-character-set=utf8mb4 < "D:\course\Javaweb\实训 8_SpringBoot3 集
成 MyBatis\db\init.sql"
```

也可以在 Navicat / IDEA Database 工具里直接执行这段 SQL。

3.2 检查是否建好

```sql
USE `xm-pro`;
SHOW TABLES;
DESC employee;
SELECT * FROM employee;
```

做完应该看到：

```
id  name      sex  no      age  description
1   thc       男   100001  18   勤学钻研，自强不息
2   zhangsan  男   100002  19   积极开拓，敢于创新
3   lisi      女   100003  23   做事细致严谨
```

> **最容易踩的坑：数据库名**
>
> 讲义和截图里用的是 xm-pro（中划线），不是 xm_pro（下划线）。

> 如果 application.yml 写成 xm_pro，MySQL 会报：
>
> Table 'xm_pro.employee' doesn't exist
>
> 因为 xm_pro 这个库可能也存在，但里面没有 employee 表。

---

4. 配置 application.yml

对应讲义：二、配置 MyBatis → 1. 配置 mybatis 实体和 xml 映射。

文件位置：springboot/src/main/resources/application.yml

```yaml
server:
  port: 8080

spring:
  datasource:
    # 注意：数据库名是 xm-pro（中划线），不是 xm_pro
    url: jdbc:mysql://localhost:3306/xm-pro?useUnicode=true&characterEncoding=utf-8&serverTimezone=Asia/Shanghai
    username: ${DB_USERNAME:root}
    password: ${DB_PASSWORD:123456}
    driver-class-name: com.mysql.cj.jdbc.Driver

mybatis:
  mapper-locations: classpath:mapper/*.xml
  configuration:
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl
    map-underscore-to-camel-case: true

pagehelper:
  helper-dialect: mysql
  reasonable: true
  support-methods-arguments: true
  params: count=countSql
```

三个 MyBatis 配置的作用：

配置	作用
mapper-locations: classpath:mapper/*.xml	告诉 MyBatis 去 resources/mapper 下找 XML
log-impl: ...StdOutImpl	在控制台打印真正执行的 SQL 和参数，方便排错
map-underscore-to-camel-case: true	数据库下划线字段自动映射成 Java 驼峰属性

> username / password 写的是 ${DB_USERNAME:root} 这种形式：

- 没有配置环境变量时，使用冒号后面的默认值 root / 123456；
- 如果本机密码不是 123456，可以改默认值，或者设置环境变量 DB_PASSWORD。

---

5. 实体类 Employee

对应讲义：10. 新建一个包：entity，新建一个实体类 Employee。

文件位置：springboot/src/main/java/com/example/entity/Employee.java

```java
package com.example.entity;

/**
```

```
 * 员工实体，字段与 employee 表一一对应。
 */
public class Employee {

    private Integer id;
    private String name;
    private String sex;
    private String no;
    private Integer age;
    private String description;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getSex() {
        return sex;
    }

    public void setSex(String sex) {
        this.sex = sex;
    }

    public String getNo() {
        return no;
    }

    public void setNo(String no) {
        this.no = no;
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }
}
```

要点：

1. 实体类属性名要和数据库字段对应：`id`、`name`、`sex`、`no`、`age`、`description`。
2. 必须有 **getter / setter**，否则 MyBatis 无法把查询结果装进对象，Jackson 也无法把对象转成 JSON。
3. `id` 和 `age` 用 `Integer`，其余用 `String`。

6. 统一返回类 Result

文件位置：springboot/src/main/java/com/example/common/Result.java

```
package com.example.common;

/**
 * 统一返回结果。
 *
 * @param <T> 返回数据的类型
 */
public class Result<T> {

    private String code;
    private String msg;
    private T data;

    public static <T> Result<T> success() {
        Result<T> result = new Result<>();
        result.setCode("200");
        result.setMsg("请求成功");
        return result;
    }

    public static <T> Result<T> success(T data) {
        Result<T> result = success();
        result.setData(data);
        return result;
    }

    public static <T> Result<T> error() {
        Result<T> result = new Result<>();
        result.setCode("500");
        result.setMsg("系统错误");
        return result;
    }

    public static <T> Result<T> error(String code, String msg) {
        Result<T> result = new Result<>();
        result.setCode(code);
        result.setMsg(msg);
        return result;
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public String getMsg() {
        return msg;
    }

    public void setMsg(String msg) {
        this.msg = msg;
    }

    public T getData() {
        return data;
    }

    public void setData(T data) {
        this.data = data;
    }
}
```

返回格式固定为：

```
{
  "code": "200",
  "msg": "请求成功",
  "data": {}
}
```

- `code`：业务状态码，`200` 成功，`500` 失败；
- `msg`：提示信息；
- `data`：真正的数据，可以是对象、集合、字符串或 `null`。

---

7. Mapper 接口

对应讲义：`3. 在包 example 内新建一个 mapper 包、4. 新建一个接口、5. 产生关联。`

文件位置：`springboot/src/main/java/com/example/mapper/EmployeeMapper.java`

```java
package com.example.mapper;

import com.example.entity.Employee;
import org.apache.ibatis.annotations.Select;

import java.util.List;

/**
 * 员工 Mapper。
 *
 * 增删改查的 SQL 写在 resources/mapper/EmployeeMapper.xml 中，
 * 方法名必须和 XML 里的 id 完全一致。
 */
public interface EmployeeMapper {

    List<Employee> selectAll();

    Employee selectById(Integer id);

    @Select("SELECT id, name, sex, no, age, description FROM employee WHERE id = #{id}")
    Employee selectByIdWithAnnotation(Integer id);

    int insert(Employee employee);

    int update(Employee employee);

    int deleteById(Integer id);
}
```

要点：

- Mapper 接口只写方法声明，不写实现；
- MyBatis 启动时用动态代理生成实现类；
- `selectByIdWithAnnotation` 用 `@Select` 注解直接写 SQL，演示“简单查询可以用注解”；
- 其余方法都去 XML 里找同名 `id` 的 SQL。

7.1 让 Spring Boot 扫描 Mapper

对应讲义：`6. 告诉 SpringBoot 如何扫描 mapper 包。`

文件位置：`springboot/src/main/java/com/example/SpringbootApplication.java`

```
package com.example;

import org.mybatis.spring.annotation.MapperScan;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
@MapperScan("com.example.mapper")
public class SpringbootApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringbootApplication.class, args);
    }
}
```

`@MapperScan("com.example.mapper")` 会扫描这个包下所有 Mapper 接口，并注册成 Spring Bean。

> 也可以在每个 Mapper 接口上写 @Mapper，两种方式选一种即可；同时写也不会报错，但没必要。

---

8. Mapper XML

对应讲义：`2. 新建 EmployeeMapper.xml 文件`、`5. namespace 关联`、`14. 写上查询语句`。

文件位置：`springboot/src/main/resources/mapper/EmployeeMapper.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.example.mapper.EmployeeMapper">

    <!-- 查询全部 -->
    <select id="selectAll" resultType="com.example.entity.Employee">
        SELECT id, name, sex, no, age, description
        FROM employee
        ORDER BY id
    </select>

    <!-- 按 id 查询 -->
    <select id="selectById" resultType="com.example.entity.Employee">
        SELECT id, name, sex, no, age, description
        FROM employee
        WHERE id = #{id}
    </select>

    <!-- 新增：useGeneratedKeys 把自增主键回填到 employee.id -->
    <insert id="insert" useGeneratedKeys="true" keyProperty="id">
        INSERT INTO employee (name, sex, no, age, description)
        VALUES (#{name}, #{sex}, #{no}, #{age}, #{description})
    </insert>

    <!-- 修改 -->
    <update id="update">
        UPDATE employee
        SET name        = #{name},
            sex         = #{sex},
            no          = #{no},
            age         = #{age},
            description = #{description}
        WHERE id = #{id}
    </update>

    <!-- 删除 -->
    <delete id="deleteById">
        DELETE FROM employee WHERE id = #{id}
```

```
    </delete>

</mapper>
```

三个核心点，必须完全对应：

XML 里的写法	必须和谁对应	写错会怎样
namespace="com.example.mapper.EmployeeMapper"	Mapper 接口的全限定类名	Invalid bound statement (not found)
<select id="selectAll">	接口方法名 selectAll	同上
resultType="com.example.entity.Employee"	实体类全限定类名	查询结果无法封装

`#{id}`、`#{name}` 是 MyBatis 的占位符，相当于 JDBC 的 `?`，会使用预编译，能防 SQL 注入。

> selectByIdWithAnnotation 用的是 @Select 注解，**不要**再在 XML 里写一个同 id 的 <select>，否则会报“Mapped Statements collection already contains value”。

---

9. Service 业务层

对应讲义：`7. 新建 service 包、12. 完善 EmployeeService.java 代码`。

文件位置：`springboot/src/main/java/com/example/service/EmployeeService.java`

```
package com.example.service;

import com.example.entity.Employee;
import com.example.mapper.EmployeeMapper;
import com.github.pagehelper.PageHelper;
import com.github.pagehelper.PageInfo;
import jakarta.annotation.Resource;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class EmployeeService {

    @Resource
    private EmployeeMapper employeeMapper;

    public List<Employee> selectAll() {
        return employeeMapper.selectAll();
    }

    public Employee selectById(Integer id) {
        return employeeMapper.selectById(id);
    }

    public Employee selectByIdWithAnnotation(Integer id) {
        return employeeMapper.selectByIdWithAnnotation(id);
    }

    public PageInfo<Employee> selectPage(Integer pageNum, Integer pageSize) {
        PageHelper.startPage(pageNum, pageSize);
        List<Employee> list = employeeMapper.selectAll();
        return PageInfo.of(list);
    }

    public int insert(Employee employee) {
        return employeeMapper.insert(employee);
    }

    public int update(Employee employee) {
        return employeeMapper.update(employee);
```

```
    }

    public int deleteById(Integer id) {
        return employeeMapper.deleteById(id);
    }
}
```

要点：

- `@Service` 把类交给 Spring 管理；
- `@Resource` 把 Mapper 代理对象注入进来；
- Controller 不直接调 Mapper，必须经过 Service；
- 现在业务简单，Service 只是中转；以后有事务、校验、计算，都写在这里。

---

10. Controller 控制层

对应讲义：`8. 新建类 EmployeeController、11. 完善 EmployeeController 的代码。`

文件位置：`springboot/src/main/java/com/example/controller/EmployeeController.java`

```
package com.example.controller;

import com.example.common.Result;
import com.example.entity.Employee;
import com.example.service.EmployeeService;
import com.github.pagehelper.PageInfo;
import jakarta.annotation.Resource;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@CrossOrigin
@RestController
@RequestMapping("/employee")
public class EmployeeController {

    @Resource
    private EmployeeService employeeService;

    /**
     * 查询全部员工。
     */
    @GetMapping("/selectAll")
    public Result<List<Employee>> selectAll() {
        return Result.success(employeeService.selectAll());
    }

    /**
     * 按 id 查询（路径参数）。
     */
    @GetMapping("/selectById/{id}")
    public Result<Employee> selectById(@PathVariable Integer id) {
        return Result.success(employeeService.selectById(id));
    }
```

```
    /**
     * 按 id 查询（查询参数，演示 @Select 注解方式）。
     */
    @GetMapping("/selectOne")
    public Result<Employee> selectOne(@RequestParam Integer id) {
        return Result.success(employeeService.selectByIdWithAnnotation(id));
    }

    /**
     * 分页查询。
     */
    @GetMapping("/selectPage")
    public Result<PageInfo<Employee>> selectPage(
            @RequestParam(defaultValue = "1") Integer pageNum,
            @RequestParam(defaultValue = "10") Integer pageSize) {
        return Result.success(employeeService.selectPage(pageNum, pageSize));
    }

    /**
     * 新增员工，id 由数据库自增。
     */
    @PostMapping("/insert")
    public Result<String> insert(@RequestBody Employee employee) {
        int rows = employeeService.insert(employee);
        return rows > 0 ? Result.success("新增成功") : Result.error("500", "新增失败");
    }

    /**
     * 修改员工，必须传 id。
     */
    @PutMapping("/update")
    public Result<String> update(@RequestBody Employee employee) {
        int rows = employeeService.update(employee);
        return rows > 0 ? Result.success("修改成功") : Result.error("500", "修改失败");
    }

    /**
     * 按 id 删除员工。
     */
    @DeleteMapping("/delete/{id}")
    public Result<String> delete(@PathVariable Integer id) {
        int rows = employeeService.deleteById(id);
        return rows > 0 ? Result.success("删除成功") : Result.error("500", "删除失败");
    }
}
```

注解解释：

注解	作用
@RestController	@Controller + @ResponseBody，返回值自动转 JSON
@RequestMapping("/employee")	给整个类加统一前缀
@GetMapping	只接收 GET 请求
@PostMapping	只接收 POST 请求
@PutMapping	只接收 PUT 请求
@DeleteMapping	只接收 DELETE 请求
@PathVariable	从 URL 路径里取参数，如 /selectById/1
@RequestParam	从 ?id=1 查询参数里取值
@RequestBody	把请求体 JSON 转成 Java 对象

完整请求链路：

```
浏览器 / Apifox
   → EmployeeController
   → EmployeeService
   → EmployeeMapper 接口
   → EmployeeMapper.xml
   → MySQL
   → 原路返回 JSON
```

---

11. 启动并测试 selectAll

对应讲义：15. 测试，输入地址 `http://localhost:8080/employee/selectAll`。

11.1 启动

在 IDEA 里运行 `SpringbootApplication`，或在 `springboot` 目录执行：

```
mvn spring-boot:run
```

控制台出现下面这行算启动成功：

```
Tomcat started on port 8080 (http)
Started SpringbootApplication in ...
```

11.2 访问

浏览器打开：

```
http://localhost:8080/employee/selectAll
```

应该返回：

```
{
  "code": "200",
  "msg": "请求成功",
  "data": [
    {
      "id": 1,
      "name": "thc",
      "sex": "男",
      "no": "100001",
      "age": 18,
      "description": "勤学钻研，自强不息"
    },
    {
      "id": 2,
      "name": "zhangsan",
      "sex": "男",
      "no": "100002",
      "age": 19,
      "description": "积极开拓，敢于创新"
    },
    {
      "id": 3,
      "name": "lisi",
      "sex": "女",
      "no": "100003",
      "age": 23,
      "description": "做事细致严谨"
    }
  ]
}
```

控制台会打印：

```
==>  Preparing: SELECT id, name, sex, no, age, description FROM employee ORDER BY id
==> Parameters:
<==    Columns: id, name, sex, no, age, description
<==        Row: 1, thc, 男, 100001, 18, 勤学钻研，自强不息
<==      Total: 3
```

如果只看到 Preparing 和 Parameters，没有 Columns/Total，说明 SQL 执行出错了，继续看下一节排错。

---

12. 单个查询

对应讲义：四、单个查询。

12.1 路径传参：`/selectById/{id}`

Mapper 接口：

```
Employee selectById(Integer id);
```

XML：

```
<select id="selectById" resultType="com.example.entity.Employee">
    SELECT id, name, sex, no, age, description
    FROM employee
    WHERE id = #{id}
</select>
```

Controller：

```
@GetMapping("/selectById/{id}")
public Result<Employee> selectById(@PathVariable Integer id) {
    return Result.success(employeeService.selectById(id));
}
```

测试：

```
http://localhost:8080/employee/selectById/1
```

返回：

```
{
  "code": "200",
  "msg": "请求成功",
  "data": {
    "id": 1,
    "name": "thc",
    "sex": "男",
    "no": "100001",
    "age": 18,
    "description": "勤学钻研，自强不息"
  }
}
```

12.2 注解方式：`@Select`

对应讲义：利用注解的方式查询单个数据。

Mapper 接口：

```
@Select("SELECT id, name, sex, no, age, description FROM employee WHERE id = #{id}")
Employee selectByIdWithAnnotation(Integer id);
```

Controller：

```
@GetMapping("/selectOne")
public Result<Employee> selectOne(@RequestParam Integer id) {
    return Result.success(employeeService.selectByIdWithAnnotation(id));
}
```

测试：

```
http://localhost:8080/employee/selectOne?id=2
```

这种方式适合简单 SQL；复杂 SQL 仍然建议写在 XML 里。

---

13. 分页查询

对应讲义：五、分页查询。

13.1 引入 PageHelper

pom.xml 里已经有：

```
<dependency>
    <groupId>com.github.pagehelper</groupId>
    <artifactId>pagehelper-spring-boot-starter</artifactId>
    <version>2.1.0</version>
    <exclusions>
        <exclusion>
            <groupId>org.mybatis</groupId>
            <artifactId>mybatis</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

13.2 Service

```
public PageInfo<Employee> selectPage(Integer pageNum, Integer pageSize) {
    PageHelper.startPage(pageNum, pageSize);
    List<Employee> list = employeeMapper.selectAll();
    return PageInfo.of(list);
}
```

关键：PageHelper.startPage(...) 必须紧挨着查询方法调用，中间不要插入别的查询。

13.3 Controller

```
@GetMapping("/selectPage")
public Result<PageInfo<Employee>> selectPage(
        @RequestParam(defaultValue = "1") Integer pageNum,
        @RequestParam(defaultValue = "10") Integer pageSize) {
    return Result.success(employeeService.selectPage(pageNum, pageSize));
}
```

13.4 测试

```
http://localhost:8080/employee/selectPage?pageNum=1&pageSize=2
```

返回里重点看这些字段：

```
{
  "code": "200",
  "msg": "请求成功",
  "data": {
    "total": 3,
    "list": [ ... ],
    "pageNum": 1,
```

```
    "pageSize": 2,
    "pages": 2,
    "isFirstPage": true,
    "isLastPage": false,
    "hasNextPage": true
  }
}
```

- total：总记录数；
- list：当前页数据；
- pages：总页数。

---

14. 增删改操作（任务 4）

对应讲义：4. MyBatis 增删改操作练习。

14.1 Mapper 接口增加三个方法

```
int insert(Employee employee);

int update(Employee employee);

int deleteById(Integer id);
```

返回值 int 表示**受影响的行数**：

- 新增成功：1；
- 修改成功：1；
- 删除成功：1；
- 没找到记录：0。

14.2 XML 增加 <insert>、<update>、<delete>

```
<!-- 新增：useGeneratedKeys 把自增主键回填到 employee.id -->
<insert id="insert" useGeneratedKeys="true" keyProperty="id">
    INSERT INTO employee (name, sex, no, age, description)
    VALUES (#{name}, #{sex}, #{no}, #{age}, #{description})
</insert>

<!-- 修改 -->
<update id="update">
    UPDATE employee
    SET name        = #{name},
        sex         = #{sex},
        no          = #{no},
        age         = #{age},
        description = #{description}
    WHERE id = #{id}
</update>

<!-- 删除 -->
<delete id="deleteById">
    DELETE FROM employee WHERE id = #{id}
</delete>
```

useGeneratedKeys="true" keyProperty="id" 的作用：

- 数据库里 id 是自增的，插入时不需要传 id；

- 插入成功后，MyBatis 会把数据库生成的主键值回填到 `employee.id`，方便后续使用。

14.3 Service 增加三个方法

```
public int insert(Employee employee) {
    return employeeMapper.insert(employee);
}

public int update(Employee employee) {
    return employeeMapper.update(employee);
}

public int deleteById(Integer id) {
    return employeeMapper.deleteById(id);
}
```

14.4 Controller 增加三个接口

```
@PostMapping("/insert")
public Result<String> insert(@RequestBody Employee employee) {
    int rows = employeeService.insert(employee);
    return rows > 0 ? Result.success("新增成功") : Result.error("500", "新增失败");
}

@PutMapping("/update")
public Result<String> update(@RequestBody Employee employee) {
    int rows = employeeService.update(employee);
    return rows > 0 ? Result.success("修改成功") : Result.error("500", "修改失败");
}

@DeleteMapping("/delete/{id}")
public Result<String> delete(@PathVariable Integer id) {
    int rows = employeeService.deleteById(id);
    return rows > 0 ? Result.success("删除成功") : Result.error("500", "删除失败");
}
```

14.5 测试新增

不要用浏览器地址栏访问 `/employee/insert`，因为浏览器默认发 GET，而新增接口要求 POST。请用 Apifox、Postman 或 IDEA 的 `requests.http`。

请求：

```
POST http://localhost:8080/employee/insert
Content-Type: application/json
{
  "name": "王五",
  "sex": "男",
  "no": "100004",
  "age": 24,
  "description": "新员工测试"
}
```

返回：

```
{"code":"200","msg":"请求成功","data":"新增成功"}
```

然后查询确认：

```
http://localhost:8080/employee/selectAll
```

14.6 测试修改

```
PUT http://localhost:8080/employee/update
Content-Type: application/json
{
  "id": 4,
  "name": "王五",
  "sex": "男",
  "no": "100004",
  "age": 25,
  "description": "修改后的描述"
}
```

返回：

```
{"code":"200","msg":"请求成功","data":"修改成功"}
```

再查 `selectById/4`，确认字段已经变化。

14.7 测试删除

```
DELETE http://localhost:8080/employee/delete/4
```

返回：

```
{"code":"200","msg":"请求成功","data":"删除成功"}
```

再查 `selectById/4`，返回：

```
{"code":"200","msg":"请求成功","data":null}
```

---

15. 完整验收清单

按顺序做完，打勾即可：

步骤	操作	预期结果
1	MySQL 执行 db/init.sql	xm-pro.employee 表存在，有 3 条数据
2	IDEA 打开 springboot，Maven 依赖下载完成	没有红色依赖报错
3	运行 SpringbootApplication	控制台出现 Tomcat started on port 8080
4	访问 /employee/selectAll	返回 code=200 和 3 条员工数据
5	访问 /employee/selectById/1	返回 id=1 的 thc
6	访问 /employee/selectOne?id=2	返回 id=2 的 zhangsan
7	访问 /employee/selectPage?pageNum=1&pageSize=2	返回 total=3, pages=2
8	POST /employee/insert	返回“新增成功”，再查能看到新记录
9	PUT /employee/update	返回“修改成功”，再查字段已改
10	DELETE /employee/delete/{id}	返回“删除成功”，再查 data=null

---

16. 常见报错排查

现象	原因	解决
Table 'xm_pro.employee' doesn't exist	数据库名写成了 xm_pro，但表在 xm-pro	改 application.yml 的 URL 为 xm-pro，或执行 db/init.sql

现象	原因	解决
Invalid bound statement (not found)	XML 的 namespace 或 id 和 Mapper 接口对不上；XML 没被加载	检查 namespace 是否为接口全限定名，方法名是否和 id 一致，mapper-locations 是否正确
浏览器 404 Whitelabel Page	Controller 没扫描到，或改完代码没重启；路径写错	确认 @RestController、@RequestMapping，重启项目
浏览器 405 Method Not Allowed	用 GET 访问了 POST/PUT/DELETE 接口	换 Apifox/Postman，选择正确请求方式
400 Bad Request	POST/PUT 没传 JSON 请求体，或 Content-Type 不对	设置 Content-Type: application/json，请求体写合法 JSON
返回 {"code":"500","msg":"系统错误"}	后端抛异常，被全局异常处理捕获	看 IDEA 控制台 ERROR 堆栈，通常会看到真实 SQL 错误
Port 8080 was already in use	端口被占用	关掉旧进程，或改 server.port
Access denied for user 'root'	数据库密码不对	改 application.yml 的 password
中文乱码	数据库/连接字符集不对	库、表用 utf8mb4，URL 带 characterEncoding=utf-8
控制台 description 显示 <<BLOB>>	LONGTEXT 类型在 MyBatis 日志里的正常显示	不影响接口返回，忽略即可
修改/删除返回“修改失败/删除失败”	传入的 id 不存在	先查 selectAll 确认 id

16.1 为什么 404 和 405 不会进 GlobalExceptionHandler？

`GlobalExceptionHandler` 处理的是 Controller 内部抛出的异常。URL 根本找不到对应方法时，Spring MVC 直接返回 404；请求方法不对时返回 405，这两种情况通常不会进入你的 `@ExceptionHandler`。

16.2 改完代码必须重启

项目没有引入 `spring-boot-devtools`，修改 Java 代码后必须重新运行 `SpringbootApplication`，否则运行的还是旧 class，常见表现就是“代码明明改了，接口还是 404/旧结果”。

---

17. 讲义核心代码解释（复习用）

17.1 Controller 逐行

```
@RestController
@RequestMapping("/employee")
public class EmployeeController {
```

- `@RestController` = `@Controller + @ResponseBody`；
- `@ResponseBody` 让方法返回值直接变成 JSON，而不是跳转页面；
- `@RequestMapping("/employee")` 给类里所有接口加统一前缀。

```
@Resource
private EmployeeService employeeService;
```

- 从 Spring 容器里找到 `EmployeeService` 对象并注入；
- Controller 不直接操作数据库，只调用 Service。

```
@GetMapping("/selectAll")
public Result selectAll() {
    List<Employee> list = employeeService.selectAll();
    return Result.success(list);
}
```

执行顺序：

1. 浏览器请求 `/employee/selectAll`；
2. Spring 找到 `selectAll()`；
3. 调用 Service；
4. Service 调用 Mapper；
5. Mapper 找到 XML 里的 `selectAll`；
6. 执行 SQL，返回 `List<Employee>`；
7. `Result.success(list)` 包装成 JSON。

17.2 Service 逐行

```
@Service
public class EmployeeService {
```

- `@Service` 把类注册成 Spring Bean；
- Controller 才能用 `@Resource` 注入它。

```
@Resource
private EmployeeMapper employeeMapper;
```

- 注入 MyBatis 生成的 Mapper 代理对象；
- Service 是 Controller 和 Mapper 之间的桥梁。

17.3 Mapper 和 XML 的对应关系

```
List<Employee> selectAll();
```

对应：

```
<select id="selectAll" resultType="com.example.entity.Employee">
    SELECT id, name, sex, no, age, description FROM employee ORDER BY id
</select>
```

- 接口方法名 `selectAll` = XML 的 `id="selectAll"`；
- `resultType` 告诉 MyBatis 每一行封装成 `Employee`；
- 多条记录自动组装成 `List<Employee>`。

17.4 分层职责

层	职责	不该做什么
Controller	接收请求、返回 JSON	不写 SQL、不直接操作数据库
Service	业务逻辑、事务、分页	不处理 HTTP 请求
Mapper	定义数据库操作方法	不写业务判断

层	职责	不该做什么
XML	写 SQL	不写 Java 逻辑

18. 提交前检查

1. `mvn clean package` 能构建成功；
2. `SpringbootApplication` 能启动；
3. 7 个接口全部实测通过；
4. 数据库表结构和实体字段一致；
5. `application.yml` 的数据库名是 `xm-pro`；
6. 代码里没有把密码等敏感信息提交到公开仓库（本机练习可保留默认值）。

完成以上内容，这次实训的 MyBatis 增删改查部分就完整做完了。